Singly Linked List – Insert At Beginning And Time Complexity.

Program:

```
#include <iostream>
#include <stack>
using namespace std;

typedef struct ListNode
{
    int data;
    struct ListNode *next;
} Node;

Node *head;

void createEmptyList(Node **head)
{
    *head = nullptr;
    cout << "Empty list created!!!" << endl;
}

void insertAtBeginning(Node **head, int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == nullptr)
    {
        cout << "Error: Memory allocation failed." << endl;
        return;
    }
    newNode->data = data;
    if (*head == nullptr)
    {
        newNode->next = nullptr;
        *head = newNode;
    }
```

```
    else
    {
        newNode->next = *head;
        *head = newNode;
    }
}
int main()
{
   createEmptyList(&head);
    while (1)
    {
        cout << "1. Insert at the beginning" << endl;
        int choice;
        cout << "Enter your choice: ";
        cin >> choice;
        switch (choice)
        {
        case 1:
        {
            int data;
            cout << "Enter the data: ";
            cin >> data;
            insertAtBeginning(&head, data);
            break;
        }
       default:
        {
            cout << "Invalid choice." << endl;
            break;
        }
        }
    }
  return 0;
}
```

Program Analysis:

```
    Node *newNode = (Node *)malloc(sizeof(Node));
```

→ *The size of newNode will be equal to the size of a Node structure and newNode is declared as a pointer to Node structure.*

→ *The size of a pointer (newNode) is typically 8 bytes on most modern 64 − bit systems, regardless of the size of the structure it points to.*

→ *The size of the Node structure itself is determined by the sum of the sizes of its members:*

→ *data(an integer) is typically 4 bytes.*

→ *next(a pointer to another Node) is typically 8 bytes.*

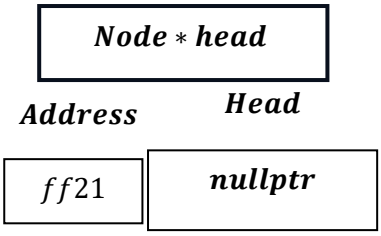
→ *Therefore, the size of the Node structure is typically 12 bytes(4 bytes for data + 8 bytes for next).*

And we already know:

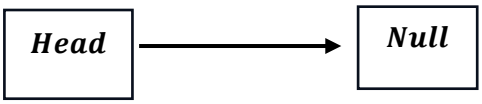
```
#include <iostream>
using namespace std;
typedef struct ListNode
{
    int data;
    struct ListNode *next;
} Node;

Node *head;

void createEmptyList(Node **head)
{
    *head = nullptr;
}

int main()
```

```
{
    createEmptyList(&head);
}
```

Head will point to nullptr , at creation of linked list.



i.e.



Now coming to insert at beginning :

```
void insertAtBeginning(Node **head, int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == nullptr)
    {
        cout << "Error: Memory allocation failed." << endl;
        return;
    }
    newNode->data = data;
    if (*head == nullptr)
    {
        newNode->next = nullptr;
        *head = newNode;
    }
```

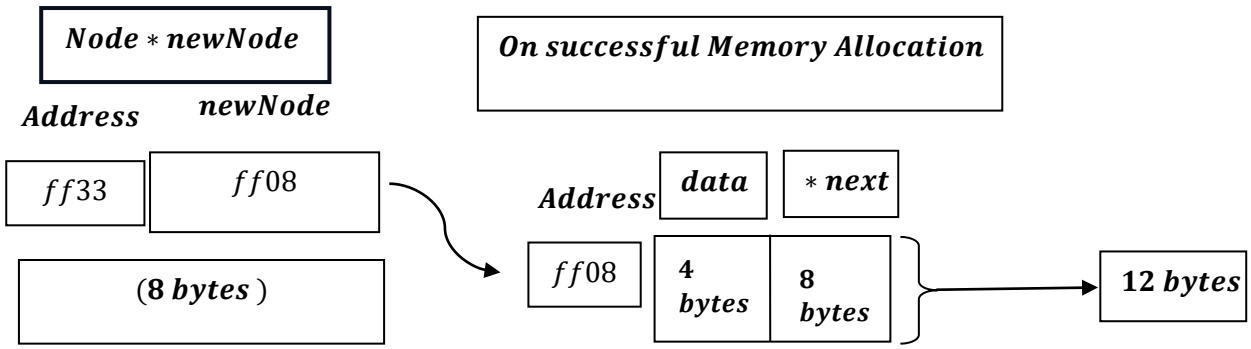
```
    else
    {
        newNode->next = *head;
        *head = newNode;
    }
}
```

```
if (newNode == nullptr)
    {
        cout << "Error: Memory allocation failed." << endl;
        return;
    }
```

What does it mean?

→ newNode will try to allocate memory for data and next pointer variable i.e. typically 8 bytes, though for variable data and structure Node's pointer variable next in total 12 bytes, but as discussed earlier , the size of a pointer (newNode) is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.

Say,



$But\ on\ Un-Successful\ memory\ allocation$:

$\rightarrow$ $if\ the\ allocation\ fails\ due\ to\ insufficient\ memory\ or\ other\ reasons, malloc\ will\ return\ a\ null\ pointer$.

$Node * newNode$	
$Address$	$newNode$
$ff33$	$nullptr$

$On\ un-successful\ Memory\ Allocation$

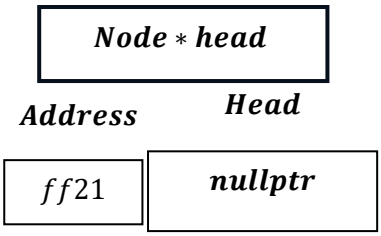
$Now, if\ the\ value\ of\ the\ pointer\ variable\ newNode\ is\ nullptr, then$:

$One\ may\ output$: `Overflow $or\ Memory\ Allocation\ Failed\ as\ a\ message$.`

Now,

```
    case 1:

        {
            int data;
            cin >> data;
            insertAtBeginning(&head, data);
            break;
        }
```

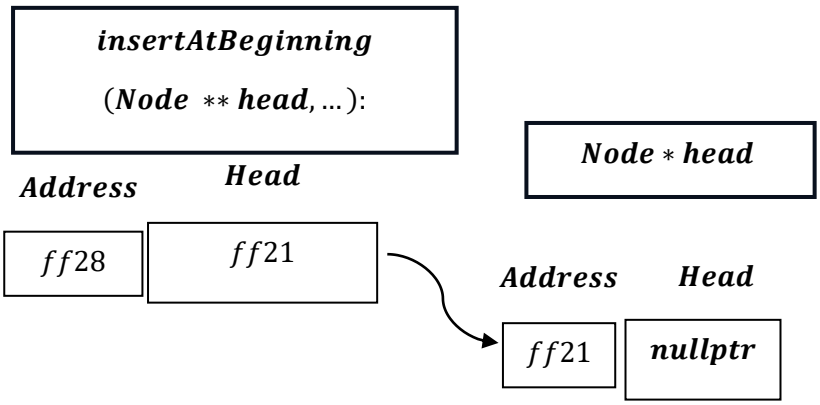
$Say\ , data = 10\ and\ currently\ head\ is\ pointed\ at\ nullptr$.



$Hence, insertAtBeginning(ff21, 10);$

```
void insertAtBeginning(Node **head, int data)
{….}
```

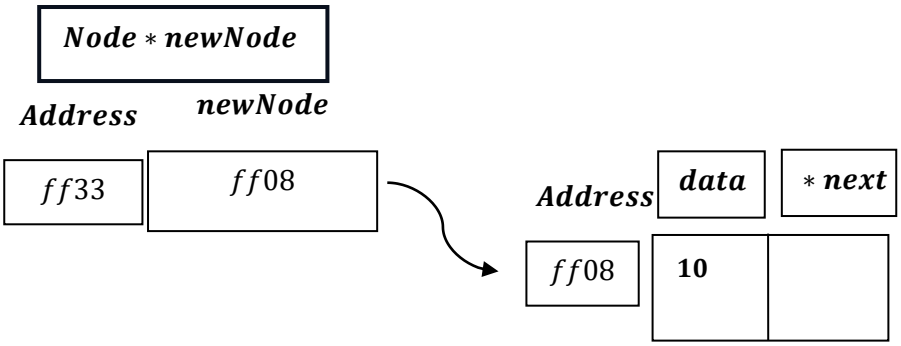
$Hence, again, we\ have\ created\ a\ temporary\ head\ variable\ pointer:$




```
    newNode->data = data;
```

$And, newNode \rightarrow data = data, i.e. newNode \rightarrow data = 10;$

$Which\ pictorially\ depicts:$

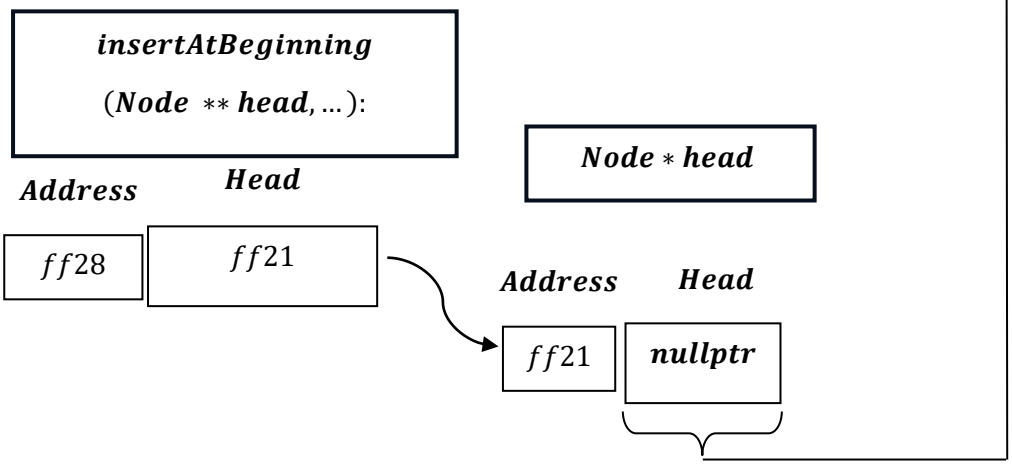


Now,

```
    if (*head == nullptr)
    {
        newNode->next = nullptr;
        *head = newNode;
    }
```

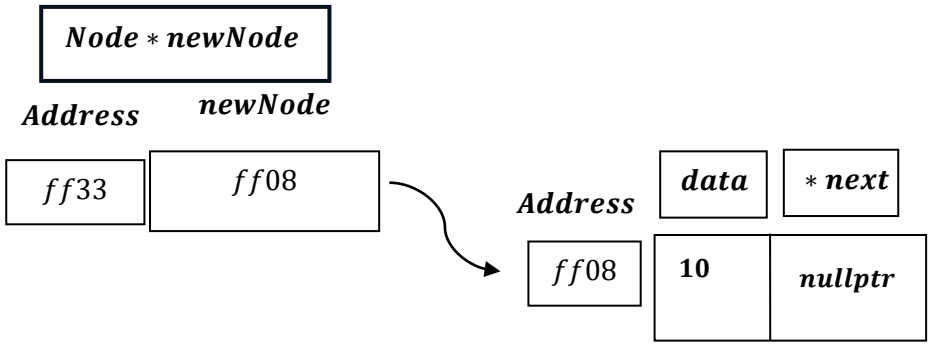
if(head == nullptr) which is true*

Why true ?
*Ans: Though * head which is of Node ** head temporary pointer to pointer variable of the function , as * head is used , it dereference from Node ** head and refer to Node * head global pointer to which Node ** head is pointing to and then its value is checked is whether equal to nullptr or not ? This occurs during execution and compilation of the insertAtBeginning function.*



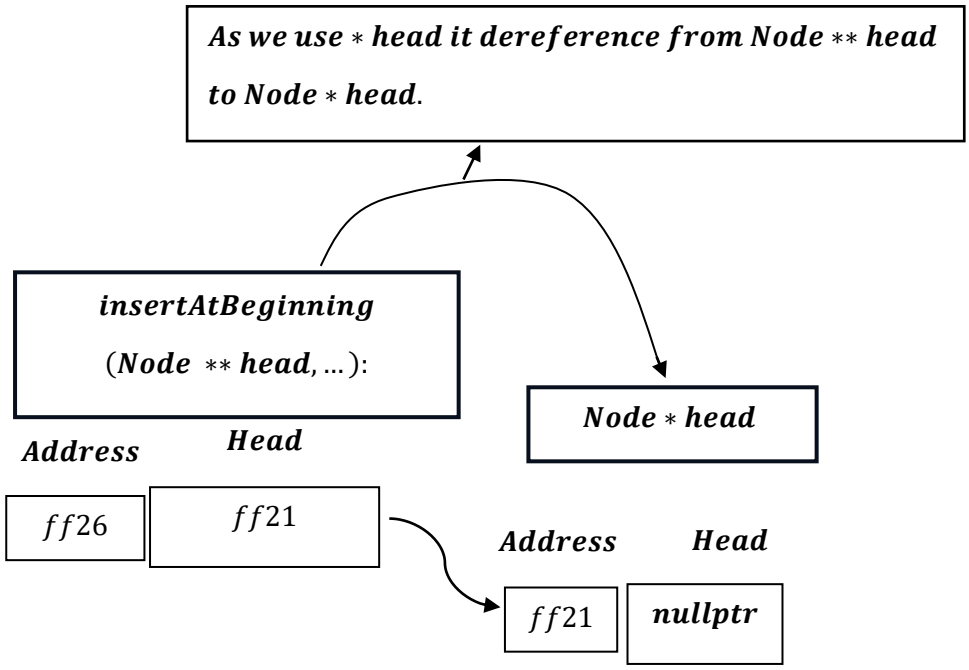
$, then:$

$newNode \rightarrow next = nullptr\ ,$

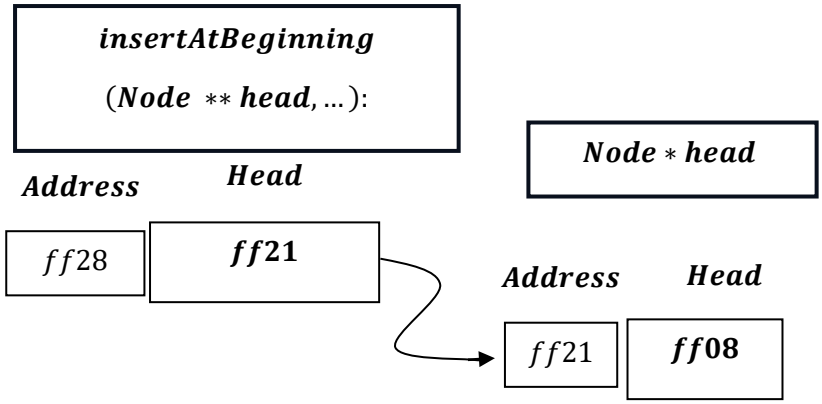


$And, * head = newNode;$

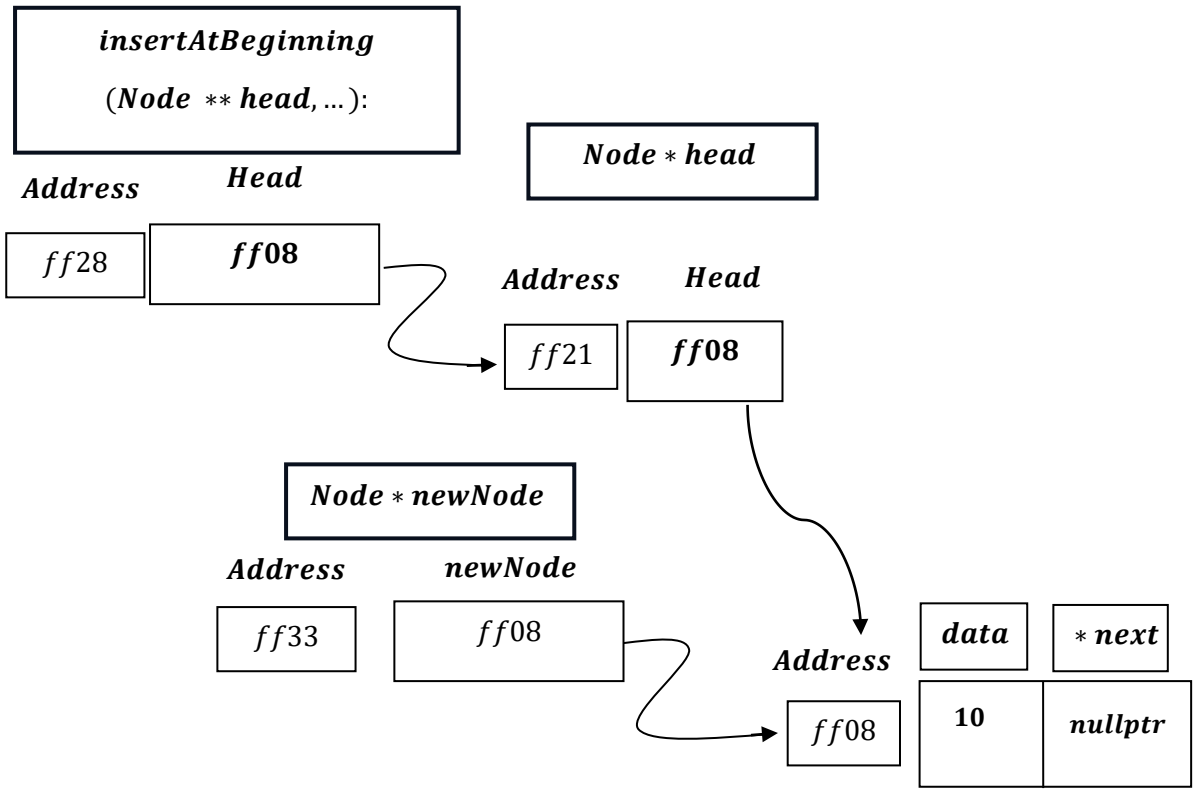
$Note\ as\ we\ are\ using\ , * head, where\ only\ head\ refers\ to\ Node\ ** head$ $which\ is\ local\ variable\ used\ in\ the\ function\ insertAtBeginning.\ Using$ $a\ pointer\ to\ head\ i.e. * head\ , will\ dereference\ from\ Node\ ** head\ and$ $refer\ us\ to\ Node * head\ to\ which\ Node\ ** head\ points\ to\ i.e.$



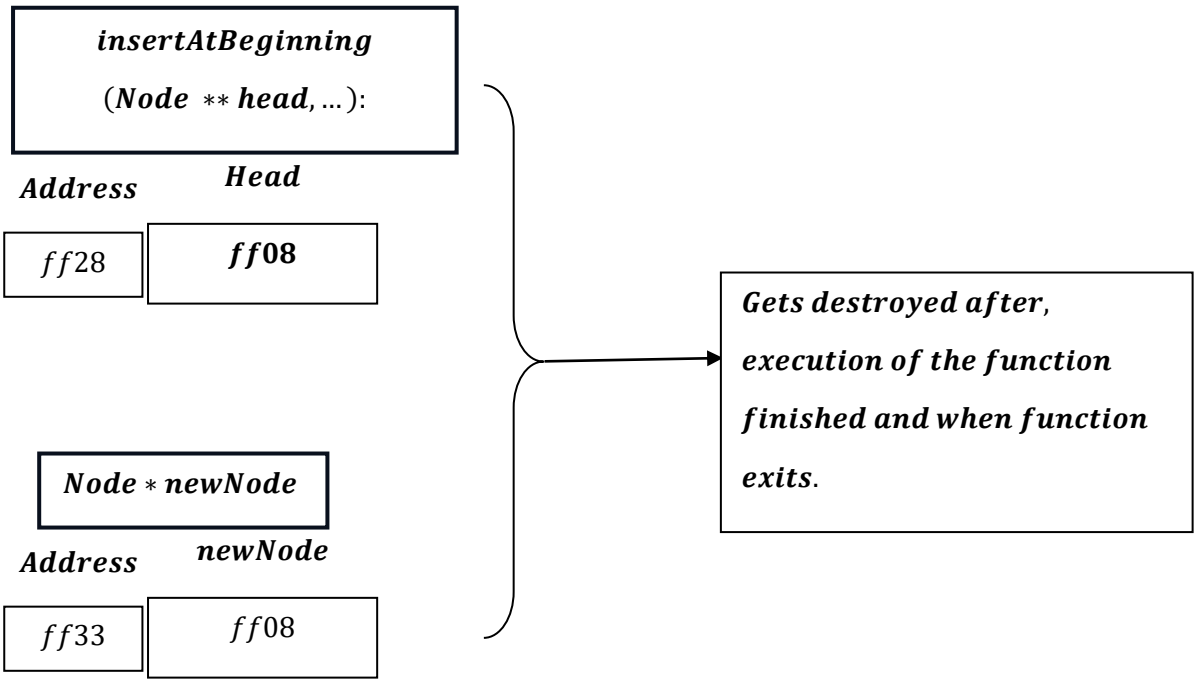
And the value i.e. the address of Node * head[here, ff21] stored in Node ** head will remain unchanged and unaltered , similarly the line of code ` * head = newNode` assigns/changes the value of global pointer Node * head to newNode's value(here , ff08)i.e. address to which newNode pointer pointing , and this happens due to dereferencing of Node ** head to Node * head, as shown below:



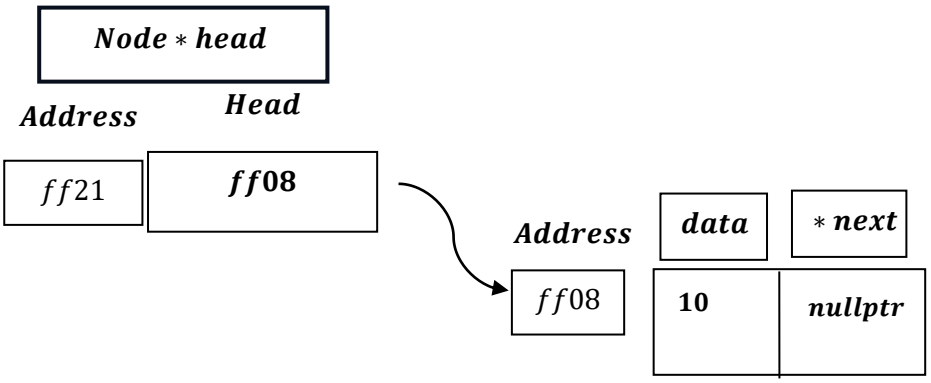
And , then Node * head starts pointing to newNode's value i.e. the address to which newNode pointer pointing to (here, ff08) .



Now, as Node * newNode and Node ** head are temporary variable, therefore gets destroyed after function execution get finished , i.e.



Then we are left with:

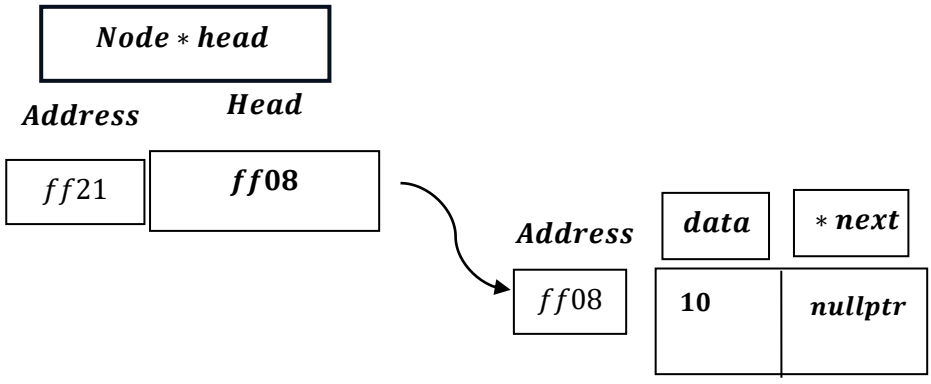


Now , lets insert another data :

```
    case 1:

        {
            int data;
            cin >> data;
            insertAtBeginning(&head, data);
            break;
        }
```

$Say, data = 12$.

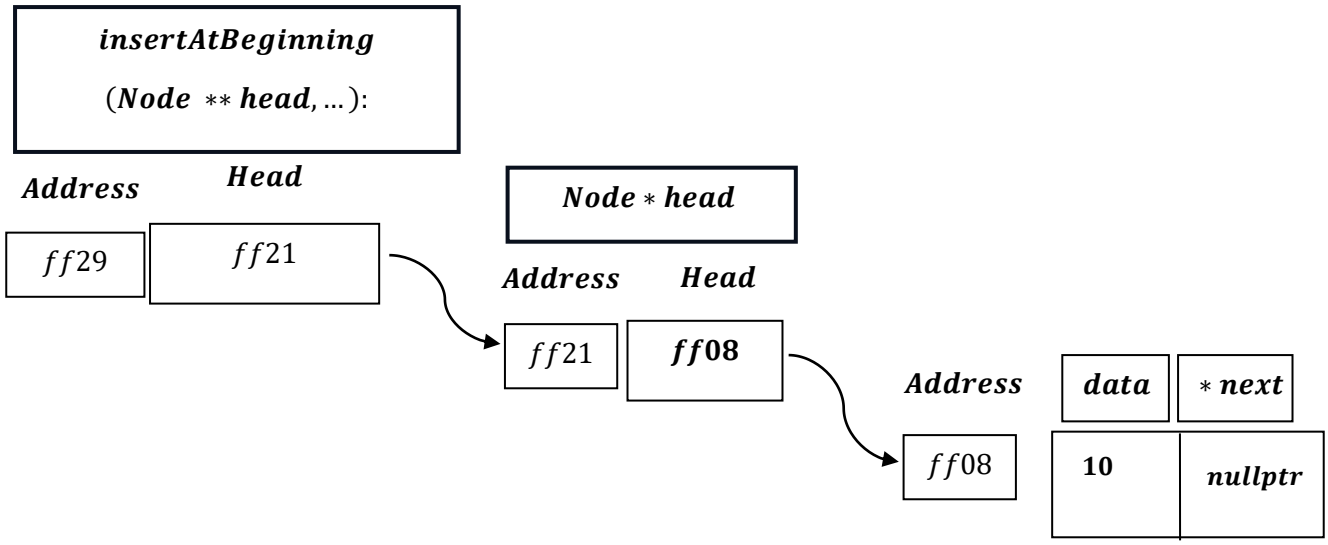
Now ,



$Hence, we\ pass$: $insertAtBeginning(ff08, 12)$.

```
void insertAtBeginning(Node **head, int data)
{….}
```

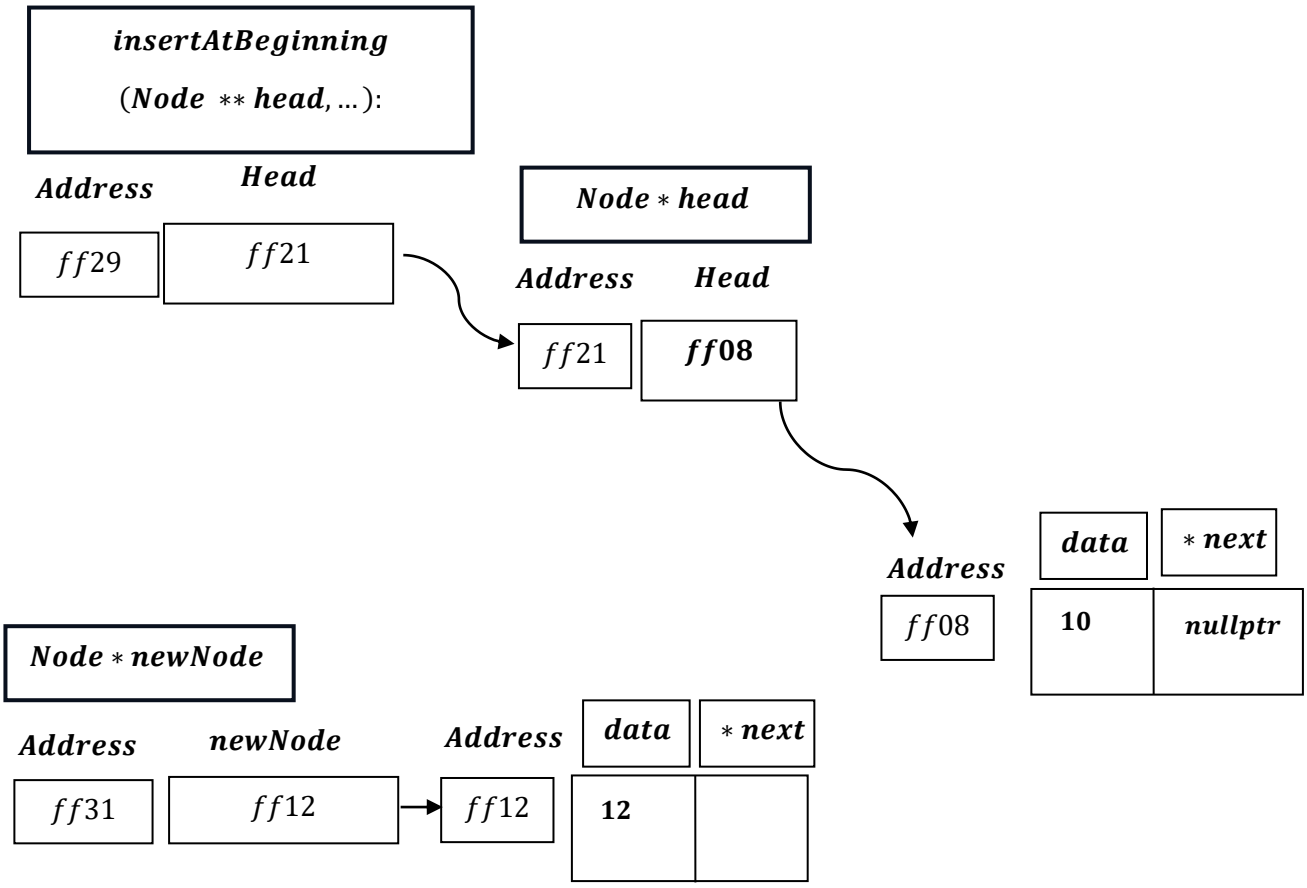
$Again$, $we\ have\ created\ a\ temporary\ head\ variable\ pointer$:




```
    newNode->data = data;
```

And, $newNode \rightarrow data = data$, *i.e.* $newNode \rightarrow data = 12$;

Which pictorially depicts:

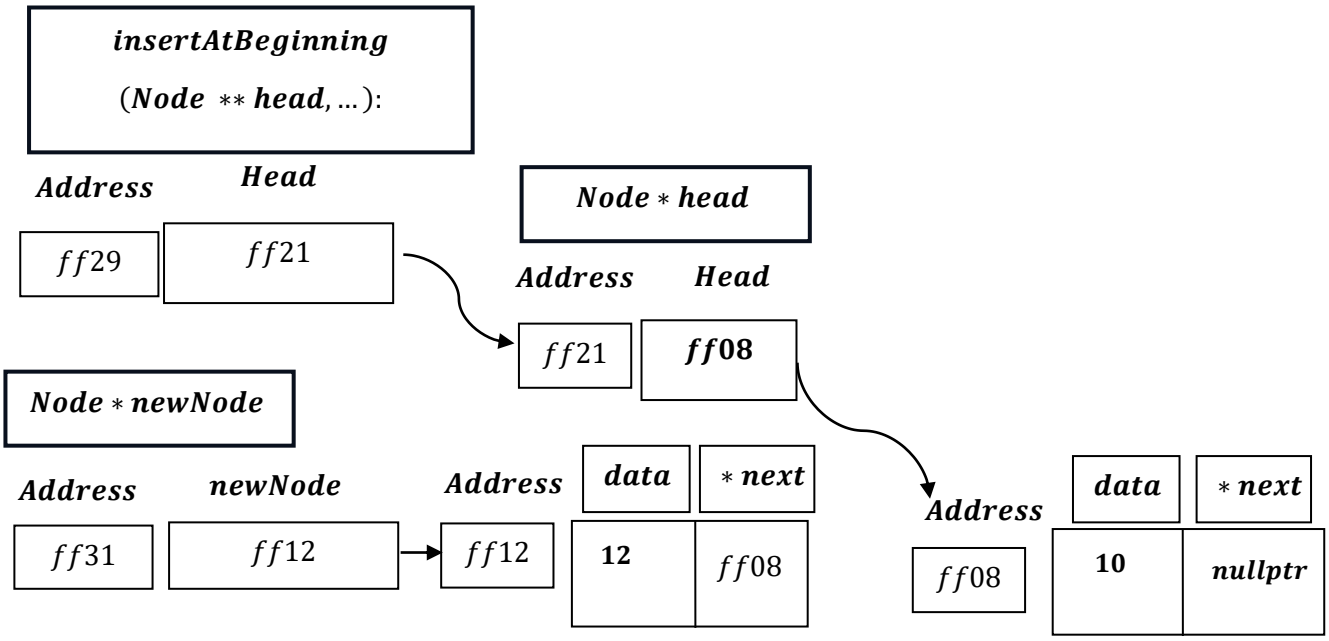


Now , head's value is not Null , contain addess , hence it will go to else part:

```
else
{
    newNode->next = *head;
    *head = newNode;
}
```

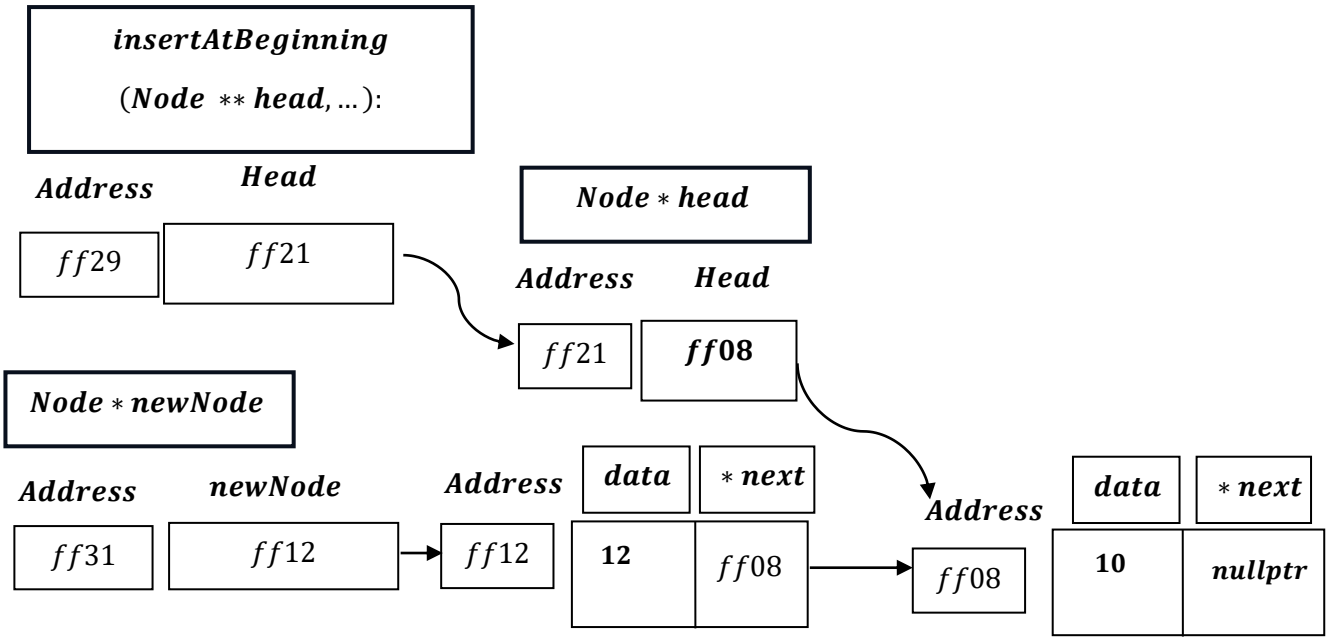
Therefore , first we get is: next pointer , member variable of structure gets assigned to address stored (here, ff08) in head pointer of Node structure.

i.e.



And therefore next pointer will start pointing to address (here, ff08).

*Again , here the same reason, newNode → next = * head , here head used is of local pointer to pointer i.e. Node ** head , and as * head is used, it will first derefer from Node ** head and refer us to Node * head , which will cause next pointer to store value contained in Node * head i.e. ff08 , address of Head node to which Node * head pointer points to.*

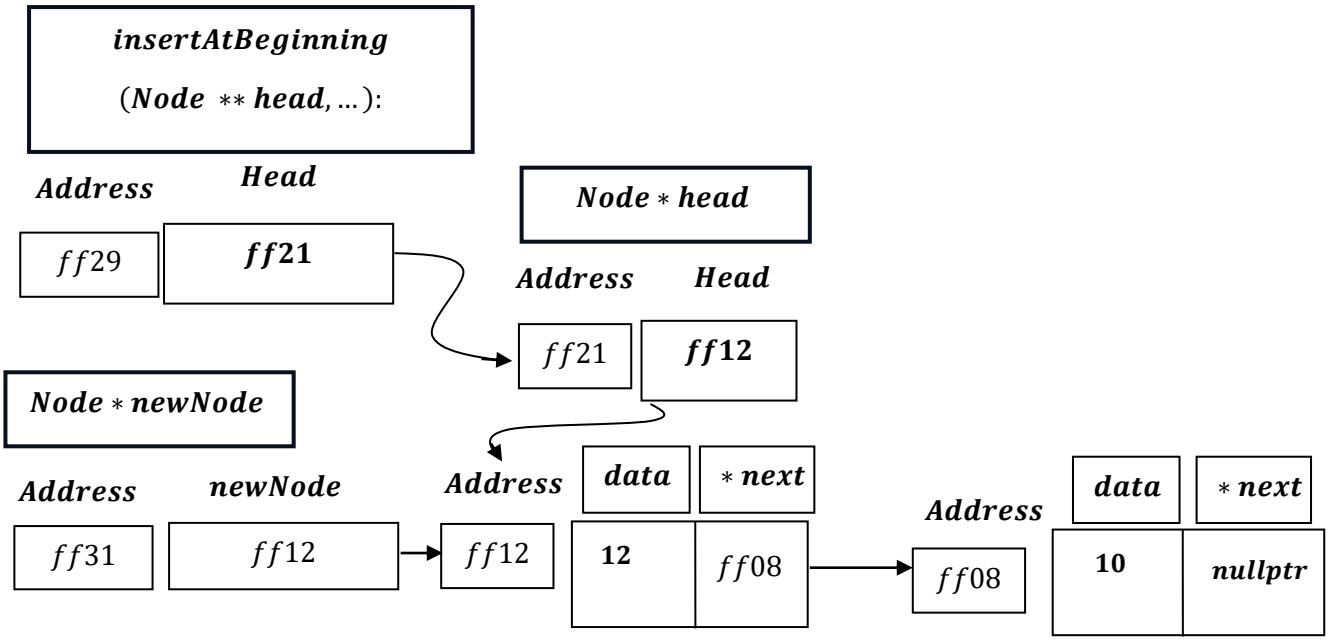


And now,

```
*head = newNode;
```

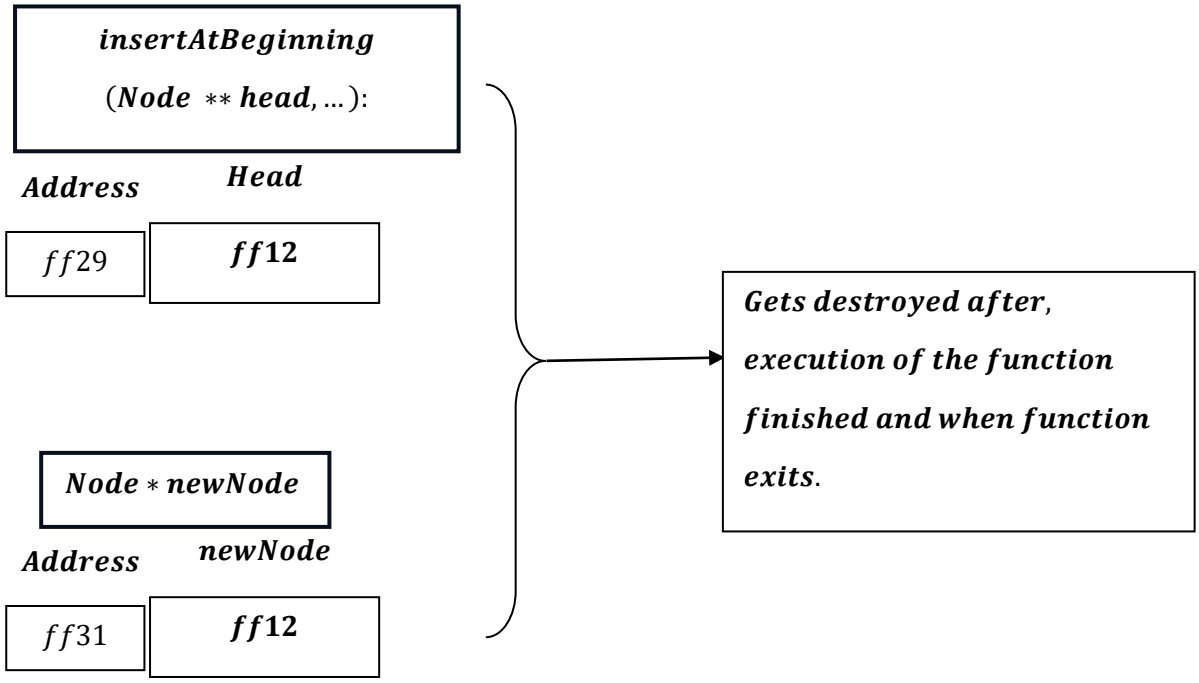
*What does it mean? It means , here head variable used is of Node ** head , temporary pointer to pointer variable in function, As we use * head , it dereference from Node ** head and refer us to Node * head, global pointer to which Node ** head points to. And the line of code ` * head = newNode` change the value of global pointer Node * head , due to dereferencing of Node ** head to Node * head and Node * head gets assigned to address of newNode pointer variable , which stored as a value in Node * head[here , ff12]. And the value i.e. the address of Node * head[here, ff21] stored in Node ** head will remain unchanged and unaltered.*

This is pictorally shown below:

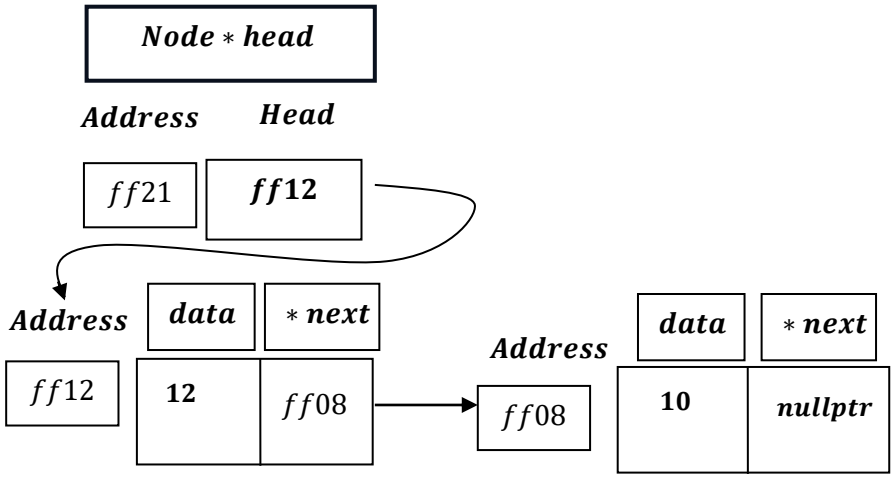


*Now, as Node * newNode and Node ** head are temporary variable, therefore gets destroyed after function execution get finished .*

i.e.



And we are left with the list:



And this process continues as we continue to insert data at bengining in Linked List.

Insert At Beginning [***Working Summary***]:

→ ***If head pointer is null i.e. list is empty then head pointer gets assigned to the address of the newly created node consist of data and next pointer which points to nullptr . That is new node becomes head of the list. And head pointer points to head of the list.***

→ ***If head pointer is not null or list is not empty, then head pointer is assigned to the address of newly created node's address and newly created node's next pointer carries address of the previous node i.e. next will point to previous node's address. That is head pointer will point to newly created node which becomes the head and next pointer of newly created node which is current head will point to previous head node's address.***

Time Complexity of the program

```
void insertAtBeginning(Node **head, int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));   → O(1)
    if (newNode == nullptr)
    {                                   → O(1)
        cout << "Error: Memory allocation failed." << endl;   → O(1)
        return;   → O(1)
    }
    newNode->data = data;   → O(1)
    if (*head == nullptr)
    {                                   → O(1)
        newNode->next = nullptr;   → O(1)

       *head = newNode;   → O(1)
    }
    else
    {
        newNode->next = *head;   → O(1)
        *head = newNode;   → O(1)
    }
}
```

→ *The function allocates memory for a new node using malloc. This operation takes constant time, typically around* $O(1)$ *on most systems.*

→ *The function checks if the new node pointer is null, indicating memory allocation failed. This comparison is a simple operation that takes constant time,* $O(1)$. *Including output statement of if and return statement takes constant time* $O(1)$.

→ *When input integer data value assigned to integer structure member variable `data` through newNode pointer of structure named Node takes constant time* $O(1)$.

→ *Checking pointer head is null takes constant amount of time i.e.* $O(1)$, *which indicates the list is empty, if empty then : the new node becomes the head and its next pointer is set to null. Both of these assignments are constant − time operations,* $O(1)$.

$\rightarrow$ ***Else if the list is non − empty then: the new node's next pointer is set to the current head, and the head pointer is updated to point to the new node. Again, these are constant − time operations,*** $O(1)$.

Hence , if we sum up i.e. $O(1) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1)$ $+O(1) = O(1)$, ***i.e. overall timecomplexity is*** $O(1)$ ***i.e. constant time complexity.***

Hence, the overall time complexity of the insertAtBeginning function is $O(1)$.

Singly Linked List	***Time Complexity***
InsertAtBeginning	$O(1)$

****